

MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 2

Software Design

Author: Andrei Radu Varga

Group: group 30433

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

April 2026

Contents

1	Problem Definition	2
2	Dependencies and Configuration	2
2.1	Dependencies (pom.xml)	2
2.2	Configuration (application.properties)	2
3	Design Phase	2
3.1	Architecture Overview	2
3.2	Design Patterns	3
3.3	RabbitMQ Configuration (Config/RabbitMQConfig.java)	3
3.4	Domain Event (Event/DomainEvent.java)	3
3.5	MongoDB Documents and Repositories (Documents/)	3
4	Implementation Phase	4
4.1	Event Publishing (Messaging/EmailEventPublisher.java)	4
4.2	Product Service Modifications (Service/ProductServiceImplementation.java)	4
4.3	Message Consumption (Messaging/EmailConsumer.java)	4
4.4	Notification Controller (Controller/NotificationController.java)	4
4.5	Notification Service (Notification/NotificationService.java)	4
4.6	Export Functionality — Strategy Pattern (Export/)	4
4.7	Product Controller Extension (Controller/ProductController.java)	5
4.8	Manage Products Panel Extension (View/ManageProductsPanel.java)	5
5	Testing and Validation	5

1 Problem Definition

This assignment extends the Online Clothes Marketplace from Assignment 1 with three new features: email notifications triggered by product events, audit logging in MongoDB, and product export in JSON, XML, and CSV formats.

The deliverables include the updated Java source code, updated pom.xml, configuration files, and this documentation.

2 Dependencies and Configuration

2.1 Dependencies (pom.xml)

Four dependencies were added:

- spring-boot-starter-data-mongodb: adds MongoDB support via MongoRepository interfaces and @Document-annotated classes.
- spring-boot-starter-amqp: adds RabbitMQ support, exposing AmqpTemplate for sending messages and @RabbitListener for consuming them.
- spring-boot-starter-mail: provides a JavaMailSender bean for sending emails over SMTP.
- jackson-dataformat-xml and jackson-datatype-jsr310: Jackson extensions for XML serialization and proper handling of LocalDateTime.

2.2 Configuration (application.properties)

Three systems need to be configured:

RabbitMQ connects to a local broker on port 5672 using the default guest credentials (works with a local Docker instance).

MongoDB is configured via spring.data.mongodb.uri pointing to a local instance, with auto-index-creation enabled.

3 Design Phase

3.1 Architecture Overview

The layered MVC architecture from Assignment 1 is kept. RabbitMQ is added as a message broker between the service layer and the notification logic. MongoDB is used only for storing event and email logs.

3.2 Design Patterns

- Strategy (Export): `ExportStrategy` is an interface with three implementations (JSON, XML, CSV). `ExportService` picks the right one at runtime based on the requested format.
- Event-Driven (Notifications): product lifecycle events are published to a RabbitMQ queue and consumed asynchronously by `EmailConsumer`.
- Data Transfer Object: `DomainEvent` is a Java record used as the message payload passed through RabbitMQ and saved to MongoDB.

3.3 RabbitMQ Configuration (`Config/RabbitMQConfig.java`)

A single durable queue named "product.events" is declared. The default RabbitMQ exchange is used, which routes messages to the queue matching the routing key, so no explicit Exchange or Binding beans are needed. A `Jackson2JsonMessageConverter` is registered for automatic JSON serialization.

3.4 Domain Event (`Event/DomainEvent.java`)

`DomainEvent` is a Java record (immutable by default). It has these fields:

- `eventId`: UUID identifying the event.
- `type`: `PRODUCT_CREATED`, `PRODUCT_UPDATED`, or `PRODUCT_DELETED`.
- `entityName`: the affected class name
- `entityId`: primary key of the affected entity.
- `userEmail`: email of the logged-in user from `UserSession`.
- `occurredAt`: timestamp of the event.

This record is both the RabbitMQ message payload and the MongoDB audit log entry. It is visible only by the admin, in the logs pane.

3.5 MongoDB Documents and Repositories (`Documents/`)

`EventLogEntry` is a `@Document` in the "event_logs" collection. One entry is saved per product event, storing all `DomainEvent` fields plus a MongoDB-generated ID.

`EmailLogEntry` is a `@Document` in the "email_logs" collection. One entry is saved per email attempt (successful or not), storing recipient, subject, body, status (SENT or FAILED), optional error message, and timestamp.

`EventLogRepository` and `EmailLogRepository` extend `MongoRepository`. Spring generates their implementations at runtime.

4 Implementation Phase

4.1 Event Publishing (Messaging/EmailEventPublisher.java)

This component does two things when it receives a `DomainEvent`:

First, it saves an `EventLogEntry` to MongoDB synchronously, so the audit record exists before the message is queued.

Second, it publishes the `DomainEvent` to RabbitMQ using the queue name as the routing key.

4.2 Product Service Modifications (Service/ProductServiceImpl.java)

`saveProduct` and `deleteProduct` now call `EmailEventPublisher.publish()` after the MySQL operation. The event type is decided at runtime: if the `ProductDto` has an ID greater than zero it is an update, otherwise it is a creation. The user email comes from `UserSession`. If no user is logged in, `userEmail` is null and no notification is sent.

4.3 Message Consumption (Messaging/EmailConsumer.java)

`EmailConsumer` has `@RabbitListener` on the "product.events" queue. Spring AMQP runs a background thread that calls the `consume` method on each message. The JSON payload is automatically deserialized into a `DomainEvent`. If `userEmail` is not blank, it delegates to `NotificationController`; otherwise the event is ignored.

4.4 Notification Controller (Controller/NotificationController.java)

A thin layer between `EmailConsumer` and `NotificationService`. It exists only to follow the architectural rule that everything must go through a controller before reaching a service.

4.5 Notification Service (Notification/NotificationService.java)

Builds a `SimpleMailMessage` from the event fields and sends it via `JavaMailSender`. Whether the send succeeds or throws, an `EmailLogEntry` is saved to MongoDB with status `SENT` or `FAILED`. The exception is caught and not rethrown, so a failed email does not requeue the RabbitMQ message or crash the listener thread.

4.6 Export Functionality — Strategy Pattern (Export/)

`ExportStrategy` is an interface with two methods: `export` (takes a list of `ProductDtos`, returns a string) and `getFormat` (returns the format name).

`JsonExportStrategy` uses Jackson's `ObjectMapper` for pretty-printed JSON.

`XmlExportStrategy` uses Jackson's `XmlMapper` with a wrapper class (`ProductListWrapper`, annotated with `@JacksonXmlRootElement`) to produce a valid XML document, since Jackson cannot serialize a bare `List` to XML.

CsvExportStrategy builds the string manually. A header row is followed by one row per product. Fields containing commas, quotes, or newlines are quoted and internal quotes are doubled, following RFC 4180.

ExportService receives all three strategy beans as a List injected by Spring, indexes them by format name in a Map, fetches products from MySQL via ProductService.getAllProducts, and delegates to the right strategy.

4.7 Product Controller Extension (Controller/ProductController.java)

ExportService was injected and an exportProducts method was added. It takes a format string and returns the serialized content.

4.8 Manage Products Panel Extension (View/ManageProductsPanel.java)

Three buttons were added: Export JSON, Export XML, Export CSV. Each opens a JFileChooser with a pre-filled filename (e.g. products.json), calls ProductController.exportProducts with the right format, and writes the result to the chosen file using FileWriter. Errors are shown with JOptionPane dialogs.

5 Testing and Validation

Testing was done manually through the GUI. The following were verified:

- Creating, updating, and deleting a product saves a DomainEvent in MongoDB's event_logs collection.
- An email is sent to the logged-in user's address and an entry with status SENT appears in email logs.
- When SMTP is unavailable, the entry gets status FAILED with the exception message, and the application continues normally.
- Each export button produces a correct file that matches the current product list.